

Functional Programming Concepts in Java: Functional Interfaces and Lambda Expressions

Introduction

Modern Java programming includes features that support a functional programming style, making code more concise, readable, and flexible. Among the most important of these features are **Functional Interfaces** and **Lambda Expressions**. These concepts simplify the way developers write code, especially when working with collections and operations that require passing behavior as parameters. This research explains functional interfaces, lambda expressions, and provides real examples of built-in functional interfaces in Java.

1. Functional Interface

Definition

A **Functional Interface** is an interface that contains **only one abstract method**.

It can have multiple default or static methods, but only one abstract method.

Key Point

Functional interfaces are used as the foundation for lambda expressions.

Example

```
@FunctionalInterface
interface MyInterface {
    void sayHello();
}
```

Explanation

- The interface has only one abstract method → so it is functional
- The annotation `@FunctionalInterface` is optional but recommended

- It ensures that the interface has exactly one abstract method
-

2. Lambda Expression

Definition

A **Lambda Expression** is a short way to implement a functional interface without creating a separate class.

General Syntax

(parameters) -> expression

Example

```
MyInterface obj = () -> {  
    System.out.println("Hello from Lambda");  
};  
obj.sayHello();
```

Explanation

- Instead of writing a full class, we use a lambda expression
 - The lambda directly provides the implementation of the method
-

Comparison: Without vs With Lambda

Without Lambda:

```
class Test implements MyInterface {  
    public void sayHello() {  
        System.out.println("Hello");  
    }  
}
```

With Lambda:

```
MyInterface obj = () -> System.out.println("Hello");
```

Benefit

- Less code
 - Easier to read
 - Faster to write
-

3. Built-in Functional Interfaces (Real Examples)

Java provides many ready-to-use functional interfaces in the package:

`java.util.function`

1. Predicate

Represents a condition (returns true or false)

```
import java.util.function.Predicate;
```

```
Predicate<Integer> isEven = x -> x % 2 == 0;  
System.out.println(isEven.test(4)); // true
```

2. Function

Takes input and returns output

```
import java.util.function.Function;
```

```
Function<Integer, Integer> square = x -> x * x;  
System.out.println(square.apply(5)); // 25
```

3. Consumer

Takes input but returns nothing

```
import java.util.function.Consumer;
```

```
Consumer<String> print = x -> System.out.println(x);  
print.accept("Hello");
```

4. Supplier

Returns a value without taking input

```
import java.util.function.Supplier;
```

```
Supplier<Double> randomValue = () -> Math.random();  
System.out.println(randomValue.get());
```

Summary Table

Interface	Input	Output	Use
Predicate	Yes	Boolean	Conditions
Function	Yes	Yes	Transform data
Consumer	Yes	No	Perform action
Supplier	No	Yes	Generate values

Conclusion

In conclusion, **Functional Interfaces** and **Lambda Expressions** are powerful features in Java that support a functional programming approach. Functional interfaces provide a simple structure with a single abstract method, while lambda expressions offer a concise way to implement these interfaces. Additionally, Java's built-in functional interfaces such as Predicate, Function, Consumer, and Supplier make it easier to perform common operations efficiently. Understanding these concepts helps developers write cleaner, shorter, and more maintainable code.
